GLAXIT

INTERNSHIP PROGRAM

Advanced Cyber Security

Developed By: Hasnain Ali

Environment: Kali Linux

Advanced Networking — VLANs, Routing & Firewalls Deep Dive

Lab Report — Simulated with Linux Network Namespaces on Kali Linux

1. Objective

Build a small routed lab in which a firewall (the Kali host) sits between two isolated networks, apply firewall rules that permit only specific traffic between them, capture and analyze the resulting traffic, and document the configuration. In place of separate virtual machines, two Linux network namespaces (ns-A and ns-B) were used as lightweight stand-ins for two hosts, connected to the host through veth (virtual Ethernet) pairs, with the host performing routing and filtering between them.

2. Lab Topology

Host / Firewall	Kali Linux (192.168.10.1 on veth-A-br, 192.168.20.1 on veth-B-br)
Network A (ns-A)	192.168.10.0/24 — namespace IP 192.168.10.2, representing the internal host
Network B (ns-B)	192.168.20.0/24 — namespace IP 192.168.20.2, representing the DMZ host
Link type	veth pairs (veth-A / veth-A-br, veth-B / veth-B-br)
Firewall policy	Default FORWARD DROP; explicit ACCEPT for TCP/22 from ns-A to ns-B, plus ESTABLISHED/RELATED

Traffic flow: ns-A → veth-A → veth-A-br → [Kali routing + iptables FORWARD chain] → veth-B-br → veth-B → ns-B.

3. Build Steps

3.1 Create the namespaces

Two namespaces were created to represent the two "hosts" in the lab:

```
sudo ip netns add ns-A
sudo ip netns add ns-B
```



Figure 1 — Creating ns-A and ns-B with sudo ip netns add.

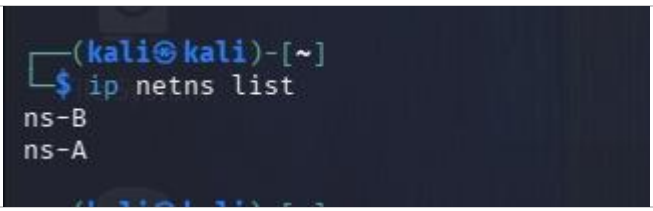


Figure 2 — ip netns list confirming both namespaces exist.

3.2 Create veth pairs (virtual cabling)

A veth pair was created for each namespace: one end stays on the host (acting as the firewall-facing interface), the other is moved into the namespace.

```
sudo ip link add veth-A type veth peer name veth-A-br
sudo ip link set veth-A netns ns-A

sudo ip link add veth-B type veth peer name veth-B-br
sudo ip link set veth-B netns ns-B
```

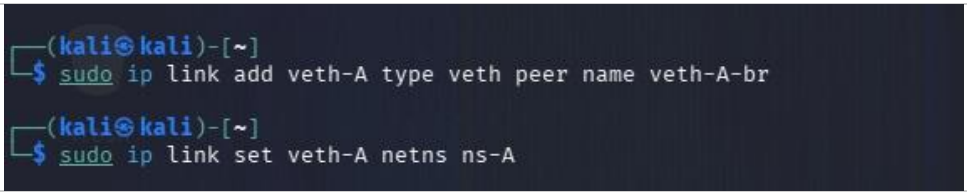


Figure 3 — veth-A / veth-A-br pair created and veth-A moved into ns-A.

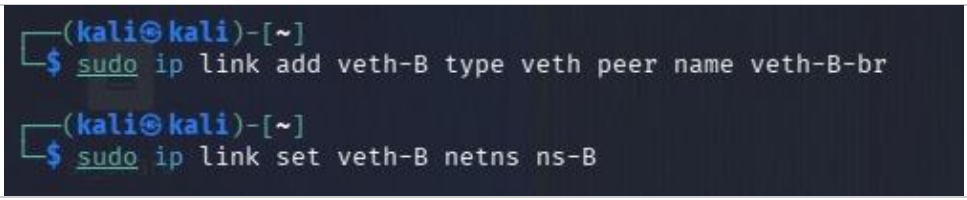


Figure 3B — veth-B / veth-B-br pair created and veth-B moved into ns-B, completing the second link.

3.3 Address the host-side interfaces

```
sudo ip addr add 192.168.10.1/24 dev veth-A-br && sudo ip link set veth-A-br up
sudo ip addr add 192.168.20.1/24 dev veth-B-br && sudo ip link set veth-B-br up
```



Figure 4 — Host-side address assigned to veth-A-br (192.168.10.1/24).



Figure 5 — Host-side address assigned to veth-B-br (192.168.20.1/24).

3.4 Address the namespace-side interfaces and set default routes

```
sudo ip netns exec ns-A ip addr add 192.168.10.2/24 dev veth-A
sudo ip netns exec ns-A ip link set veth-A up
sudo ip netns exec ns-A ip link set lo up
sudo ip netns exec ns-A ip route add default via 192.168.10.1
```

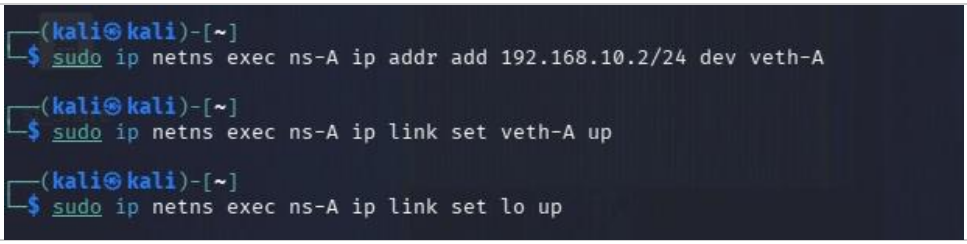


Figure 6 — ns-A given IP 192.168.10.2/24, interface and loopback brought up.



Figure 7 — Default route added inside ns-A, pointing at the host (192.168.10.1).

The same steps were applied to ns-B in a single sequence — address, interface up, loopback up, and default route via 192.168.20.1:

```
sudo ip netns exec ns-B ip addr add 192.168.20.2/24 dev veth-B
sudo ip netns exec ns-B ip link set veth-B up
sudo ip netns exec ns-B ip link set lo up
sudo ip netns exec ns-B ip route add default via 192.168.20.1
```

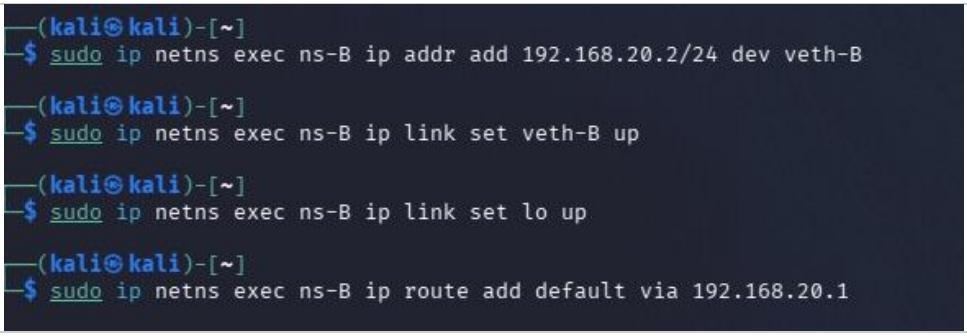


Figure 8 — Full ns-B configuration: address, link up, loopback up, and default route.

3.5 Verify interface state

ip link show was run on the host first, then against each namespace, to confirm the veth interfaces were present end-to-end and to sanity-check their reported state:

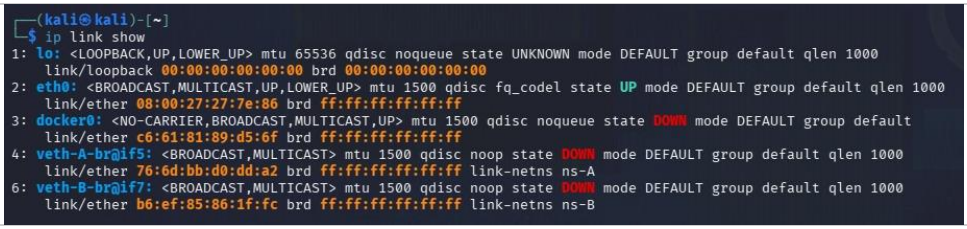


Figure 8C — Host-side view (ip link show): eth0 and docker0 are the host's normal interfaces; veth-A-br@if5 and veth-B-br@if7 are the firewall-facing ends of the two veth pairs, linked to netns ns-A and ns-B respectively.



Figure 9 — ns-A interfaces (lo, veth-A).



Figure 10 — ns-B interfaces (lo, veth-B).

Note: interfaces reported state DOWN in these views — on the host side and in both namespaces — even after the "ip link set … up" commands and despite connectivity working correctly in later steps (Section 3.6). This is a known cosmetic quirk of noqueue/veth interfaces reported through ip link show rather than an indication of a real fault — the ping and curl tests below confirm the links were functionally up.

3.6 Confirm host-to-namespace connectivity

```
sudo ip netns exec ns-A ping -c 2 192.168.10.1
sudo ip netns exec ns-B ping -c 2 192.168.20.1
```

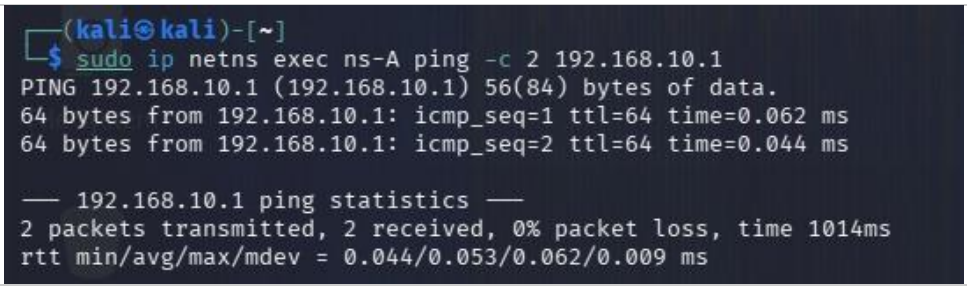


Figure 11 — ns-A successfully pings its gateway, 192.168.10.1 (0% loss).

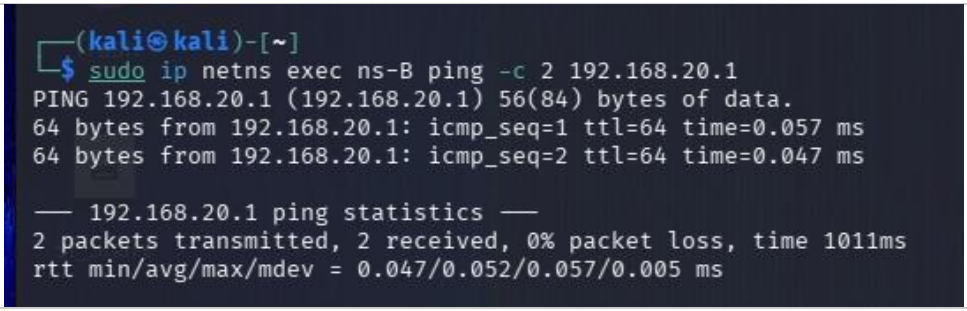


Figure 12 — ns-B successfully pings its gateway, 192.168.20.1 (0% loss).

3.7 Enable IP forwarding on the host

For the host to act as a router between the two namespaces (rather than just a dead-end for each), IPv4 forwarding must be turned on in the kernel:

```
sudo sysctl -w net.ipv4.ip_forward=1
```



Figure 12B — IPv4 forwarding enabled on the host (net.ipv4.ip_forward = 1). This is what allows packets to be routed between ns-A and ns-B through the host at all, before any iptables rules are even considered.

4. Firewall Configuration and Testing

4.1 Cross-network reachability before the firewall rule set was in its final state

```
sudo ip netns exec ns-A ping -c 2 192.168.20.2
```

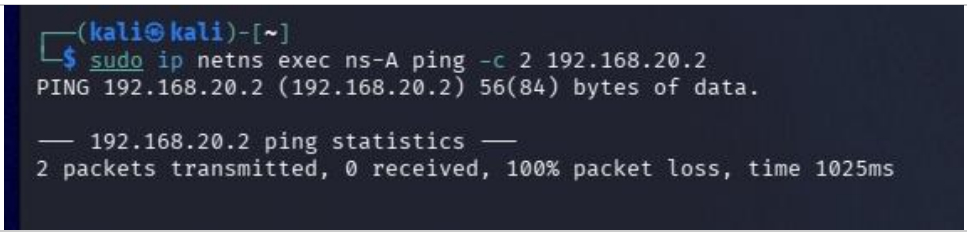


Figure 13 — ns-A → ns-B ping: 100% packet loss, confirming the default-deny FORWARD policy blocks unsolicited cross-network traffic.

4.2 Apply the firewall rules

With the default FORWARD policy set to DROP, two explicit ACCEPT rules were added: one permitting ns-A to reach ns-B on TCP/22 only, and one permitting the return traffic for any already-established connection.

```
sudo iptables -P FORWARD DROP
sudo iptables -A FORWARD -s 192.168.10.2 -d 192.168.20.2 -p tcp --dport 22 -j ACCEPT
sudo iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT
```

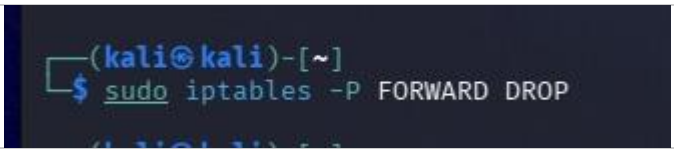


Figure 13B — Default FORWARD policy set to DROP: every packet crossing between ns-A and ns-B is now blocked unless a later rule explicitly allows it.

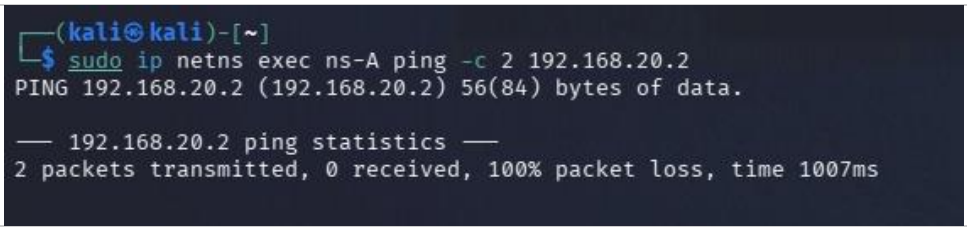


Figure 13C — Re-confirming the block immediately after setting the DROP policy: ns-A → ns-B still shows 100% packet loss, before either ACCEPT rule has been added.

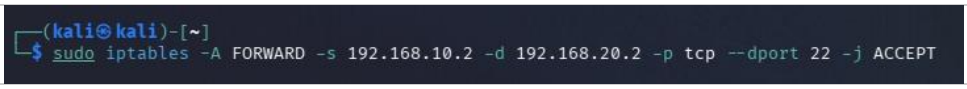


Figure 14 — Rule permitting ns-A → ns-B on TCP/22 only.

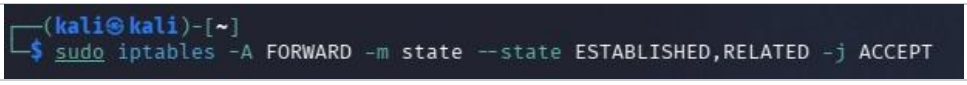


Figure 15 — Rule permitting established/related return traffic.

4.3 Stand up a test service in ns-B

A simple HTTP listener was started on port 22 inside ns-B to give the allowed rule something concrete to test against:

```
sudo ip netns exec ns-B python3 -m http.server 22 &
```



Figure 16 — Test HTTP service listening on port 22 inside ns-B.

4.4 Test the allowed path

```
sudo ip netns exec ns-A curl -m 3 192.168.20.2:22
```

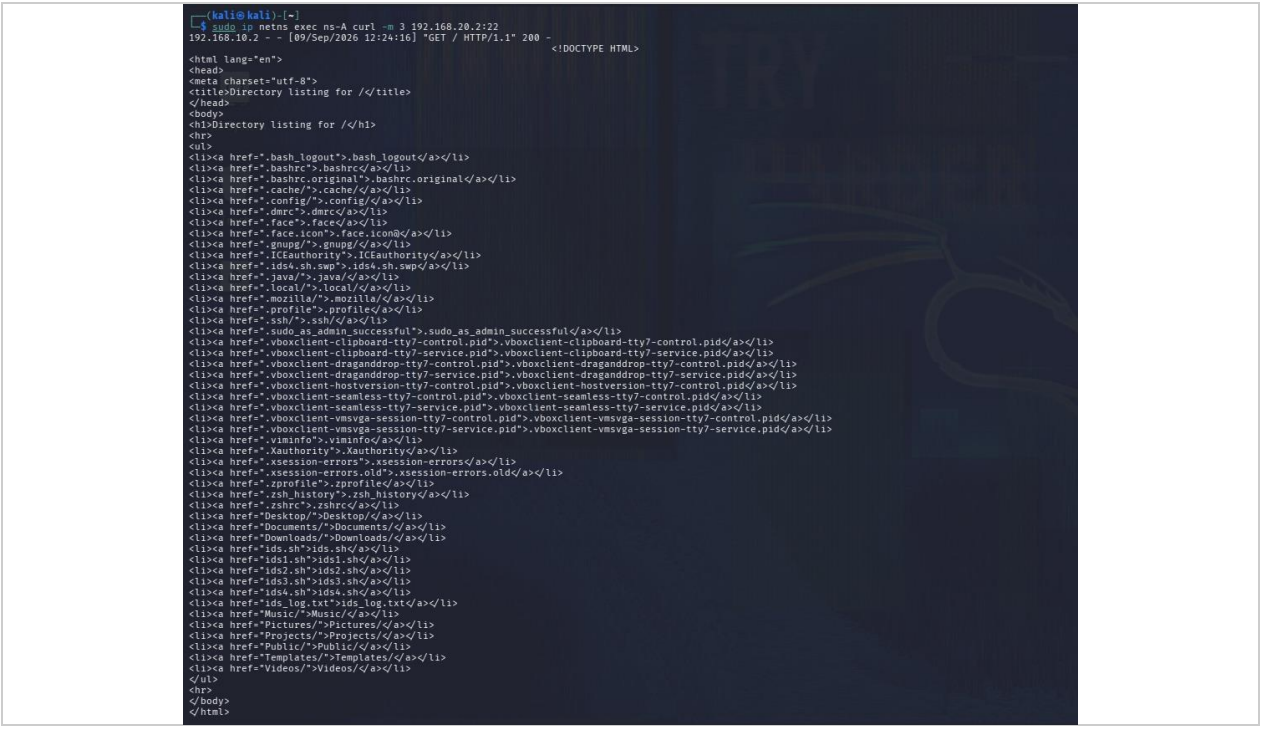


Figure 17 — ns-A successfully reaches ns-B on the allowed port (22): the request is logged by the http.server and a 200 OK directory listing is returned to curl, confirming the ACCEPT rule works as intended.

4.5 Test a blocked path

As a negative control, the same request was repeated against a port with no matching ACCEPT rule (8080):

```
sudo ip netns exec ns-A curl -m 3 192.168.20.2:8080
```

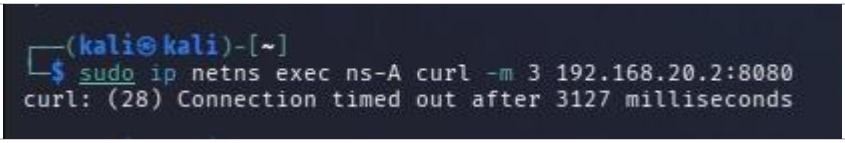


Figure 17B — Connection to the unpermitted port 8080 times out after ~3 seconds (curl error 28), confirming the default-DROP policy correctly blocks everything not explicitly allowed.

5. Traffic Capture and Analysis

Traffic on the host-side interface facing ns-A (veth-A-br) was captured with tcpdump while the tests above were run:

```
sudo tcpdump -i veth-A-br -w capture.pcap
```



Figure 18 — tcpdump capturing on veth-A-br; 4 packets captured, 0 dropped.

The capture was then opened in Wireshark for inspection:

```
wireshark capture.pcap
```



Figure 18B — Launching Wireshark produced a non-fatal GUI warning ("Failed to register with host portal... Connection already associated with an application ID"). This is a desktop-integration message from the Linux xdg-desktop-portal layer, unrelated to packet capture — Wireshark opened and analyzed capture.pcap normally despite it.

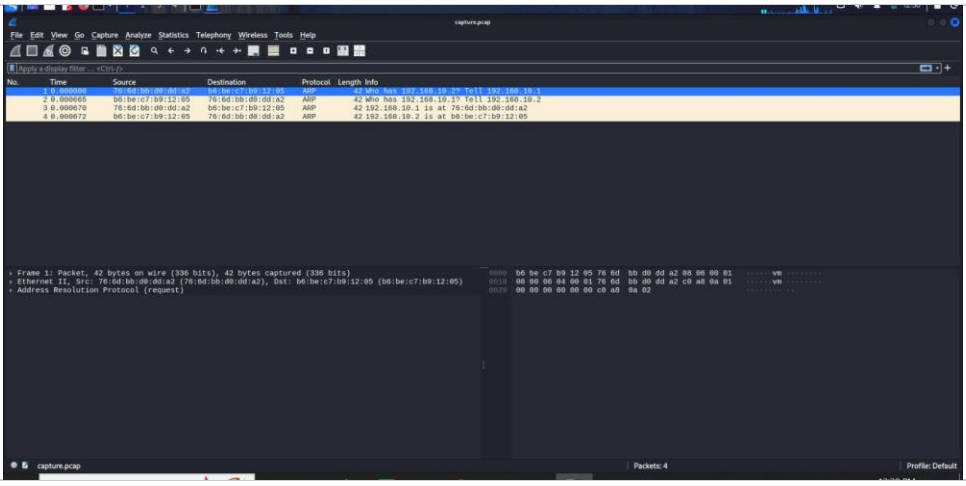


Figure 19 — Wireshark view of capture.pcap.

Analysis: the 4 captured frames are an ARP request/reply exchange between ns-A (76:6d:bb:d0:dd:a2) and the host bridge interface (b6:be:c7:b9:12:05) — "Who has 192.168.10.2? Tell 192.168.10.1" and the corresponding reply pair in both directions. This is address resolution happening before the TCP/22 conversation, confirming the two ends of the link were correctly resolving each other's MAC addresses at the data-link layer prior to the successful curl request in Figure 17.

6. Firewall Ruleset Export

```
sudo iptables-save > firewall_rules.txt
```



Figure 20 — Exporting the active ruleset to firewall_rules.txt for submission.

7. Summary of Findings

- Two isolated namespaces (ns-A, ns-B) were successfully connected through the Kali host using veth pairs, simulating a two-network topology without needing separate VMs.

- With no firewall rules in place, cross-namespace traffic is naturally isolated once a default DROP policy is set on the FORWARD chain (Figure 13).
- A targeted ACCEPT rule for TCP/22 (plus an ESTABLISHED/RELATED rule for return traffic) allowed exactly the intended traffic through, while a request to an unpermitted port (8080) timed out — confirming the firewall enforces both the allow and the default-deny side of the policy (Figures 13B–17B).
- Packet capture confirmed normal ARP resolution between the two ends of the link prior to the successful TCP session (Figures 18–19).
- The final ruleset was exported to firewall_rules.txt as the auditable record of the configuration (Figure 20).

8. Deliverables

- This report (network topology, configuration steps, and traffic analysis notes).
- firewall_rules.txt — exported iptables ruleset.
- capture.pcap — packet capture referenced in Section 5.
- Topology diagram (to be produced in draw.io per the task deliverable — see Section 2 for the addressing/topology this should reflect).